

Eksamen i BED-1304/BED-1504 Python-Lab

Fagansvarlig: Markus J. Aase

Kontinuasjonseksamen 2026

Informasjon (til studenter)

Dette er kontinuasjonseksamen i BED-1304/BED-1504, Python-lab. Den består av tre deler.

- **Del 1:** 40 flervalgsoppgaver. Ett riktig svar per oppgave. Dere får ikke minus-poeng av å svare feil.
- **Del 2:** 6 kortvarsoppgaver som tester kodeforståelse, feilsøking og konseptforklaring. Svar kort og presist i Del 2 (maks 4–8 linjer per spørsmål).
- **Del 3:** 2 programmeringsoppgaver hvor du skal skrive Python-kode. Under eksamen, i WiseFlow, har dere ikke mulighet til å teste koden. Dette tas hensyn til i sensoreringen. Sensor er først og fremst opptatt av forståelse. Derfor anbefales på det sterkeste at dere kommenterer koden deres!

Hjelpemidler

Det er **ikke tillatt** med hjelpemidler under denne eksamenen.

Lykke til!

Del 1 – Flervalgsoppgaver (Maks 40 poeng)

1. Hva skriver `print(type(7))` ut?
 - a) `<class 'float'>`
 - b) `<class 'str'>`
 - c) `<class 'int'>`
 - d) `<class 'bool'>`
2. Hva er resultatet av `10 / 4`?
 - a) 2
 - b) 2.5
 - c) 2.0
 - d) 3
3. Hva gjør operatoren `//`?
 - a) Flyttallsdivisjon
 - b) Heltallsdivisjon
 - c) Restdivisjon
 - d) Eksponentiering
4. Hvordan lager du en tom dictionary?
 - a) `d = []`
 - b) `d = ()`
 - c) `d = {}`
 - d) `d = dict[]`
5. Hva returnerer `len('Python')`?
 - a) 5
 - b) 6
 - c) 7
 - d) Error
6. Hva gjør `np.sum([1,2,3])`?
 - a) 6
 - b) 5
 - c) 3
 - d) Error
7. Hva returnerer en funksjon uten `return`?
 - a) 0
 - b) False
 - c) None
 - d) Tom streng

8. Hvordan definerer man en funksjon som heter `f` med `x` som argument?

- a) `function f(x);`
- b) `def f(x):`
- c) `define f[x]:`
- d) Alle alternativ over er korrekt.

9. Hva gjør `np.argsort(a)` gitt en array `a`?

- a) Sorterer arrayen `a` in-place
- b) Returnerer indeksene som sorterer `a`
- c) Returnerer et sortert array av verdier
- d) Gir en feilmelding hvis `a` ikke er en array.

10. Vi har brukt biblioteket `pandas` i kurset. Hva brukes det primært til?

- a) Til å arbeide med data på en tabulær form
- b) Til å jobbe med numerisk matematikk
- c) Til å lage egne funksjoner
- d) Til å finne likevektspunktet mellom tilbud og etterspørsel.

11. Hvordan bytter du verdiene til `x` og `y` uten midlertidig variabel?

- a) `x = y; y = x`
- b) `x, y = y, x`
- c) `swap(x,y)`
- d) `tmp = x; x = y; y = tmp`

12. Hva gjør `plt.plot(x,y)`?

- a) Lager linjeplot av `y` mot `x`
- b) Lager linjeplot av `x` mot `y`
- c) Lager scatterplot
- d) Lager boksplott

13. Hva blir resultatet av koden under?

```
1 for i in range(3):  
2     print(i)
```

- a) 1, 2, 3
- b) 0, 1, 2
- c) 0, 1, 2, 3
- d) 1, 2

14. Hvordan henter du ut det første elementet i en liste kalt `liste`?

- a) `liste[1]`
- b) `liste(0)`
- c) `liste[0]`

- d) `liste.first()`
15. Hva skjer hvis du prøver å addere en streng og et tall, f.eks. `'5' + 5`?
- a) Du får 10
 - b) Du får strengen `'55'`
 - c) Du får en `TypeError`
 - d) Python konverterer automatisk tallet til en streng
16. Hvilken funksjon i NumPy brukes for å lage en sekvens av tall, f.eks. fra 0 til 10?
- a) `np.range(10)`
 - b) `np.arange(11)`
 - c) `np.seq(0, 10)`
 - d) `np.list(10)`
17. Hva er forskjellen på en liste og en NumPy-array når du ganger med 2 (f.eks. `x * 2`)?
- a) Ingen forskjell, begge doubler verdiene.
 - b) Listen doubler verdiene, arrayen repeterer elementene.
 - c) Listen repeterer elementene, arrayen doubler verdiene.
 - d) Begge gir en feilmelding.
18. Hva gjør funksjonen `df.head()` i Pandas?
- a) Viser de første 5 radene i en `DataFrame`
 - b) Viser navnet på alle kolonnene
 - c) Viser statistisk sammendrag av dataene
 - d) Sletter den første raden
19. Hvordan velger du kolonnen **"Alder"** fra en `DataFrame` kalt `df`?
- a) `df.get("Alder")`
 - b) `df["Alder"]`
 - c) `df(Alder)`
 - d) `df.column("Alder")`
20. Hva brukes `pd.csv('fil.csv')` til?
- a) Å skrive data til en CSV-fil
 - b) Å lese data fra en CSV-fil inn i en `DataFrame`
 - c) Å konvertere en Excel-fil til CSV
 - d) Å plote data fra en CSV-fil
21. I Matplotlib, hvilken funksjon brukes for å sette tittel på x-aksen?
- a) `plt.title()`
 - b) `plt.xlabel()`
 - c) `plt.axis_x()`
 - d) `plt.label(x)`

22. Hva må du importere for å kunne bruke symbolske variabler i Python (slik vi har gjort i kurset)?
- a) `import numpy as np`
 - b) `import pandas as pd`
 - c) `import sympy as sp`
 - d) `import math`
23. Hvordan definerer du et symbol x i SymPy?
- a) `x = sp.symbols('x')`
 - b) `x = 'x'`
 - c) `x = sp.var(x)`
 - d) `sp.define('x')`
24. Hva gjør `sp.diff(f, x)` i SymPy?
- a) Integrerer funksjonen f med hensyn på x
 - b) Deriverer funksjonen f med hensyn på x
 - c) Løser ligningen $f = 0$ for x
 - d) Finner differansen mellom f og x
25. Hva returnerer `sp.solve(x**2 - 4, x)`?
- a) 2
 - b) [-2, 2]
 - c) 4
 - d) `x2 = 4`
26. Hva er formålet med funksjonen `np.random.seed(42)`?
- a) Å generere et helt tilfeldig tall hver gang
 - b) Å sørge for at vi får reproduserbare *tilfeldige* tall
 - c) Å kryptere dataene
 - d) Å nullstille alle variabler
27. Hvis du skal simulere et terningkast (heltall 1-6), hvilken funksjon er mest passende?
- a) `np.random.randint(1, 7)`
 - b) `np.random.uniform(1, 6)`
 - c) `np.random.normal(1, 6)`
 - d) `np.random.choice([1, 7])`
28. Hva betyr innrykk i Python?
- a) Det er kun for å gjøre koden lesbar.
 - b) Det definerer hvilke linjer som tilhører en blokk (f.eks. inni en funksjon eller løkke).
 - c) Det brukes for å kommentere ut kode.
 - d) Det spiller ingen rolle i Python.

29. Hva er en `boolean` verdi?
- a) Et heltall
 - b) En tekststreng
 - c) En verdi som enten er `True` eller `False`
 - d) Et desimaltall
30. Hvilken operator brukes for å sjekke om to variabler er like?
- a) `=`
 - b) `==`
 - c) `!=`
 - d) `<>`
31. Hva skjer i koden `if x > 10:` hvis `x` er 5?
- a) Koden inne i if-blokken kjøres.
 - b) Koden inne i if-blokken hoppes over.
 - c) Programmet gir en feilmelding.
 - d) Programmet avsluttes umiddelbart.
32. Hvordan legger du til elementet 4 på slutten av listen `a = [1, 2, 3]`?
- a) `a.add(4)`
 - b) `a.append(4)`
 - c) `a.insert(4)`
 - d) `a + 4`
33. Hva vil `np.mean(arr)` beregne gitt en array `arr`?
- a) Summen av elementene
 - b) Medianen av elementene
 - c) Gjennomsnittet av elementene
 - d) Standardavviket
34. Hva gjør metoden `df.describe()` i Pandas?
- a) Tegner en graf av dataene
 - b) Gir en oppsummering med statistikk (antall, snitt, min, maks osv.)
 - c) Beskriver datatypene i hver kolonne
 - d) Sorterer dataene stigende
35. Hva betyr feilmeldingen `NameError: name 'x' is not defined`?
- a) Variabelen `x` er brukt uten at den har blitt tildelt en verdi først
 - b) Variabelen `x` er av feil datatype
 - c) Du har glemt et kolon
 - d) Du prøver å dele på null

36. Hva er slicing i Python, f.eks. `tekst[0:3]`?
- a) Å dele teksten i to like deler
 - b) Å hente ut en del av sekvensen fra indeks 0 til (men ikke med) 3
 - c) Å reversere teksten
 - d) Å fjerne de 3 første tegnene
37. Hvilket bibliotek er `plt` en vanlig forkortelse for?
- a) `plotly`
 - b) `pandas`
 - c) `matplotlib.pyplot`
 - d) `seaborn`
38. I en funksjonsdefinisjon, hva kalles variablene i parentesene? F.eks. `def min_funksjon(x, y):`
- a) Globale variabler
 - b) Parametere (eller argumenter)
 - c) Return-verdier
 - d) Konstanter
39. Hva er riktig måte å konvertere strengen `'123'` til et heltall?
- a) `str("123")`
 - b) `float("123")`
 - c) `int("123")`
 - d) `list("123")`
40. Hvorfor bruker vi ofte NumPy fremfor vanlige lister når vi jobber med store tallmengder?
- a) NumPy bruker mindre minne og er raskere til matematiske operasjoner
 - b) NumPy har færre funksjoner enn lister
 - c) Lister kan ikke inneholde desimaltall
 - d) NumPy er innebygd i Python og krever ingen import

Del 2 – Kortsvar (Maks 30 poeng)

Svar kort og presist (4–8 linjer).

1. Forklar forskjellen på *mutable* og *immutable* datatyper i Python. Gi eksempel på hver type. (Maks 3 poeng)
2. Hva er hensikten med `for`- og `while`-løkker? Gi eksempel på hvordan hver av dem kan brukes. (Maks 6 poeng)
3. Forklar hva følgende kode gjør (Maks 3 poeng):

```
1 x = [1, 2, 3]
2 y = x
3 y.append(4)
4 print(x)
```

4. Finn feilen(e) i koden og skriv korrekt versjon (Maks 6 poeng):

```
1 def budsjett(lonnn, utgift)
2     penger_igjen = utgift - lonnn
3     return penger_igjen
4
5 lonnn = 30000
6 utgift = 25000
7 print(f'Hvis jeg tjener {lonnn} kr, og har {utgift} kr i utgift - da sitter
    jeg igjen med {penger_igjen}.')
```

5. Forklar kort hva koden under gjør her. (Maks 6 poeng)

```
1 import pandas as pd
2 import numpy as np
3
4 df = pd.DataFrame({
5     "aksje": ["Equinor", "DNB", "Telenor", "Yara", "Norsk Hydro"],
6     "kurs": [254.673, 183.948, 112.337, 402.881, 71.256]
7 })
8
9 df["kurs_rundet"] = df["kurs"].round(1)
10
11 df.head()
```

6. **Kodeforståelse:** Hva gjør koden under? Forklar linje for linje hva som skjer i løkken, og hva som til slutt skrives ut.

```
1 tall_liste = [10, 5, 8, 12, 3]
2 resultat = 0
3
4 for x in tall_liste:
5     if x > 8:
6         resultat += x
7     elif x % 2 != 0:
8         resultat -= x
9     else:
10        resultat += 1
11
12 print(resultat)
```

Del 3: Programmeringsoppgaver

I denne oppgaven skal dere skrive Python-kode etter beste evne. Kommenter koden deres godt for å vise hvordan dere tenker.

Du har ikke mulighet til å teste koden, og vi vil derfor se forbi noen små syntaksfeil. Likevel skal du skrive kode som er så korrekt at den ville kunne kjøres i Python.

Lykke til!

Oppgave 1 – Sparing med årlig innskudd

Du har tidligere laget et program for banken som beregner rente, men nå ønsker kundene å spare mer aktivt. De vil sette inn et fast beløp hvert år i tillegg til startkapitalen.

Du får i oppdrag å oppdatere programmet ditt for å håndtere dette.

Skriv et Python-program som gjør følgende:

- Lag en funksjon `sparing_med_innskudd()` som tar inn fire parametere: `startkapital`, `arlig_innskudd`, `rente` og `sparemal`.
- Funksjonen skal bruke en `while`-løkke for å finne ut hvor mange år det tar før saldoen på kontoen er større eller lik sparemålet.
- **Viktig:** I løkken skal saldoen øke med renten *først*, og deretter legges det årlige innskuddet til.
- Funksjonen skal returnere antall år (heltall) det tar å nå målet.
- Til slutt skal du kalle på funksjonen med verdiene under, og printe ut svaret med en beskrivende tekst.

- `startkapital = 5 000` kr
- `arlig_innskudd = 2 000` kr
- `rente = 0.04` (4%)
- `sparemal = 30 000` kr

Husk å kommentere koden!

Oppgave 2 — Analyse av eksamensresultater

Du har fått jobb som dataansvarlig ved et universitet. Du har fått en CSV-fil med resultater fra en eksamen, og skal finne ut hvem som har bestått og hvem som ikke har det.

Du laster inn filen slik:

```
1 import pandas as pd
2
3 df = pd.read_csv('data/karakterer.csv')
4 df.head()
```

Datarammen `df` ser slik ut, med kolonnene `'Student'` og `'Poeng'`:

Student	Poeng
Ola	45
Kari	82
Per	35
Lise	39
Espen	90

Universitetet har bestemt at grensen for å bestå er **40 poeng**.

Gjør følgende oppgaver:

1. Lag en ny kolonne i **df** som heter **'Bestått'**. Denne kolonnen skal inneholde **True** hvis studenten har 40 poeng eller mer, og **False** hvis de har mindre enn 40 poeng.
2. Bruk en **for**-løkke til å iterere gjennom datarammen, og lag en liste som inneholder **navnene** på de studentene som *ikke* har bestått (altså de som har under 40 poeng).
3. Print listen med navn til slutt.

HINT:

Under er en oversikt over noen potensielt nyttige funksjoner i Python:

Funksjon / metode	Forklaring
<code>len(df)</code>	Returnerer antall rader i en DataFrame.
<code>range(len(df))</code>	Lager en sekvens indekser fra 0 til siste rad. Nyttig i for-løkker.
<code>df.loc[i, 'Kolonne']</code>	Henter verdien i rad i og kolonne 'Kolonne'.
<code>df['Ny'] = df['Gml'] >= x</code>	Lager en ny kolonne med True/False basert på en sjekk.
<code>liste.append(verdi)</code>	Legger til en verdi på slutten av en liste.
<code>print(f'tekst {var}')</code>	F-string for å printe tekst blandet med variabler som her heter var .

Løsningsforslag

Del 1 — Flervalgsoppgaver (sensorveiledning)

Her listes riktige svar (bokstav) for hver oppgave i Del 1 i samme rekkefølge som oppgavene.

1. c) `<class 'int'>`
2. b) 2.5
3. b) Heltallsdivisjon
4. c) `d = {}`
5. b) 6
6. a) 6
7. c) None
8. b) `def f(x):`
9. b) Returnerer indeksene som sorterer `a`
10. a) Til å arbeide med data på en tabulær form
11. b) `x, y = y, x`
12. a) Lager linjeplot av `y` mot `x`
13. b) 0, 1, 2
14. c) `liste[0]`
15. c) Du får en `TypeError`
16. b) `np.arange(11)`
17. c) Listen repeterer elementene, arrayen dobler verdiene.
18. a) Viser de første 5 radene i en `DataFrame`
19. b) `df["Alder"]`
20. b) Å lese data fra en CSV-fil inn i en `DataFrame`
21. b) `plt.xlabel()`
22. c) `import sympy as sp`
23. a) `x = sp.symbols('x')`
24. b) Deriverer funksjonen f med hensyn på x
25. b) `[-2, 2]`
26. b) Å sørge for at vi får reproduserbare tilfeldige tall
27. a) `np.random.randint(1, 7)`
28. b) Det definerer hvilke linjer som tilhører en blokk

29. c) En verdi som enten er `True` eller `False`
30. b) `==`
31. b) Koden inne i if-blokken hoppes over.
32. b) `a.append(4)`
33. c) Gjennomsnittet av elementene
34. b) Gir en oppsummering med statistikk
35. a) Variabelen `x` er brukt uten at den har blitt tildelt en verdi først
36. b) Å hente ut en del av sekvensen fra indeks 0 til (men ikke med) 3
37. c) `matplotlib.pyplot`
38. b) Parametere (eller argumenter)
39. c) `int("123")`
40. a) NumPy bruker mindre minne og er raskere til matematiske operasjoner

Del 2 — Kortsvar (løsningsforslag)

1. Forskjellen på *mutable* og *immutable* datatyper i Python:

En **mutable** (foranderlig) datatype er et objekt som kan endres etter at det er opprettet (f.eks. legge til elementer eller endre verdier på en indeks). En **immutable** (uforanderlig) datatype kan ikke endres etter opprettelse; skal man **endre** verdien, må man i praksis opprette et helt nytt objekt.

Eksempler:

- **Mutable:** lister ('list'), ordbøker ('dict'), 'np.array'.
- **Immutable:** heltall ('int'), flyttall ('float'), strenger ('str'), tupler ('tuple').

```

1 # Mutable eksempel (liste):
2 liste = [1, 2, 3]
3 liste[0] = 100           # Dette er lov, endrer listen direkte (in-place)
4
5 # Immutable eksempel (streng):
6 tekst = "Hei"
7 # tekst[0] = "h"         # Dette gir feilmelding (TypeError)!
```

2. Hva er hensikten med for- og while-løkker? Gi eksempel på hvordan hver av dem kan brukes.

Hensikten med løkker er å repetere en blokk med kode flere ganger effektivt, uten å måtte skrive samme kode om igjen (automatisering av gjentakelser).

- **for-løkke:** Brukes primært når man vet på forhånd hvor mange ganger koden skal kjøres, eller når man skal iterere gjennom hvert element i en sekvens (som en liste, streng eller `range`).
- **while-løkke:** Brukes når man vil repetere koden så lenge en bestemt logisk betingelse er sann (`True`). Denne passer best når man ikke vet nøyaktig antall repetisjoner på forhånd.

Eksempler:

```
1 # For-løkke eksempel (itererer et bestemt antall ganger):
2 for i in range(5):
3     print(f"Runde nummer {i}")
4
5 # While-løkke eksempel (kjører til betingelse brytes):
6 teller = 0
7 while teller < 5:
8     print(f"Teller er {teller}")
9     teller += 1 # Viktig å øke telleren for å unngå evig løkke
```

3. Forklar hva følgende kode gjør:

```
1 x = [1, 2, 3]
2 y = x
3 y.append(4)
4 print(x)
```

Koden skriver ut: [1, 2, 3, 4]

Forklaring: Når man skriver `y = x`, lages det *ikke* en kopi av listen. I stedet settes `y` til å peke på det samme liste-objektet i minnet som `x` (begge variablene refererer til samme liste). Siden lister er *mutable* (foranderlige), vil endringen som gjøres via `y` (`append`) også være synlig når vi printer `x`.

4. Finn feilen(e) i koden og skriv korrekt versjon (Maks 6 poeng): Feil som må rettes:

- (a) **Syntaksfeil:** Mangler kolon (`:`) etter funksjonsdefinisjonen.
- (b) **Logisk feil:** Regnestykket `utgift - lønn` vil gi et negativt tall (eller gjelden). For å finne penger igjen bør det være `lønn - utgift`.
- (c) **Syntaksfeil:** Mangler avsluttende parentes `)` i `print`-funksjonen.
- (d) **Scope/Manglende funksjonskall:** Variabelen `penger_igjen` er lokal inne i funksjonen. Den finnes ikke der `print` kalles. Funksjonen `budsjett` blir aldri kalt. Man må kalle funksjonen og lagre resultatet i en variabel.

Korrekt versjon:

```
1 def budsjett(lønn, utgift):                # La til :
2     penger_igjen = lønn - utgift          # Endret rekkefølge (Logisk feil)
3     return penger_igjen
4
5 lønn = 30000
6 utgift = 25000
7
8 # Må kalle funksjonen for å gjøre beregningen
9 resultat = budsjett(lønn, utgift)
10
11 # La til ) og bruker variabelen 'resultat'
12 print(f'Hvis jeg tjener {lønn} kr, og har {utgift} kr i utgift - da sitter
    jeg igjen med {resultat}.')
```

5. Forklar kort hva koden under gjør her. (Maks 6 poeng) Forklaring:

- (a) Koden importerer bibliotekene `pandas` (som `pd`) og `numpy` (som `np`).
- (b) Det opprettes en `DataFrame` (`df`) med to kolonner: "aksje"(selskapsnavn) og kurs (flyttall/desimaltall).
- (c) Linjen `df['kurs_rundet'] = ...` oppretter en **ny kolonne** i tabellen kalt 'kurs_rundet'. Verdiene i denne kolonnen er tallene fra 'kurs'-kolonnen avrundet til én desimal.
- (d) `df.head()` viser de 5 første radene av DataFramen (i dette tilfellet vises hele tabellen siden den kun har 5 rader).

6. Hva gjør koden under? Forklar linje for linje hva som skjer i løkken, og hva som til slutt skrives ut.

Utskrift: 15

Forklaring linje for linje:

- Vi starter med `tall_liste = [10, 5, 8, 12, 3]` og `resultat = 0`.
- **Runde 1 (x=10):** Sjekker if `10 > 8` (Sant). Legger 10 til resultatet. `resultat` er nå 10.
- **Runde 2 (x=5):** Sjekker if `5 > 8` (Usant). Går til `elif 5 % 2 != 0` (Sant, 5 er oddetall). Trekker 5 fra resultatet. `resultat` er nå $10 - 5 = 5$.
- **Runde 3 (x=8):** Sjekker if `8 > 8` (Usant). Går til `elif 8 % 2 != 0` (Usant, 8 er partall). Går til `else`. Legger til 1. `resultat` er nå $5 + 1 = 6$.
- **Runde 4 (x=12):** Sjekker if `12 > 8` (Sant). Legger 12 til resultatet. `resultat` er nå $6 + 12 = 18$.
- **Runde 5 (x=3):** Sjekker if `3 > 8` (Usant). Går til `elif 3 % 2 != 0` (Sant). Trekker 3 fra resultatet. `resultat` er nå $18 - 3 = 15$.
- **Print:** Skriver ut verdien 15.

Del 3 — Programmeringsoppgaver (løsningsforslag)

Oppgave 1 – Sparing med årlig innskudd

Nedenfor er et løsningsforslag for hvordan man kan løse oppgaven. Det er viktig at studenten viser forståelse for `while`-løkken og rekkefølgen på rente og innskudd.

```
1 # Definerer funksjonen med parametere for startkapital, innskudd, rente og mål
2 def sparing_med_innskudd(startkapital, arlig_innskudd, rente, sparemal):
3     # Initialiserer variabler
4     saldo = startkapital
5     ar = 0
6
7     # While-løkke som kjører så lenge saldo er mindre enn sparemålet
8     while saldo < sparemal:
9         # Legger til renter først (saldo øker med renta først)
10        saldo = saldo * (1 + rente)
11        # Legger deretter til det årlige innskuddet
12        saldo = saldo + arlig_innskudd
13        # Øker telleren for antall år
14        ar += 1
15
16    # Returnerer antall år
17    return ar
18
19 # Kaller funksjonen med verdiene fra oppgaven
```

```

20 aar_til_mal = sparing_med_innskudd(5000, 2000, 0.04, 30000)
21
22 # Printer resultatet med en f-string
23 print(f"Det tar {aar_til_mal} år å nå sparemålet på 30 000 kr.")

```

Oppgave 2 — Analyse av eksamensresultater

I denne oppgaven testes studenten i bruk av Pandas for databehandling og enkle løkker for filtrering.

- `df['Bestått'] = ...` brukes for å lage den logiske kolonnen.
- Oppgaven ber spesifikt om en **for**-løkke for å lage listen over de som ikke besto (selv om dette teknisk sett kunne vært gjort med filtrering, skal løkke-ferdigheter testes her).

```

1 import pandas as pd
2
3 # (Simulerer innlesing av data slik at koden er kjørbar i løsningsforslaget)
4 data = {
5     "Student": ["Ola", "Kari", "Per", "Lise", "Espen"],
6     "Poeng": [45, 82, 35, 39, 90]
7 }
8 df = pd.DataFrame(data)
9 # I eksamensbesvarelsen vil studenten typisk ha: df = pd.read_csv(...)
10
11 # 1. Lager ny kolonne 'Bestått' (True hvis poeng >= 40, ellers False)
12 df['Bestått'] = df['Poeng'] >= 40
13
14 # 2. Bruker en løkke for å finne studenter som IKKE bestod eksamen
15 ikke_bestatt_liste = []
16
17 # Itererer gjennom indekser (eller rader)
18 for i in range(len(df)):
19     # Sjekker om poengsummen er under 40
20     if df.loc[i, 'Poeng'] < 40:
21         # Henter navnet og legger til i listen
22         navn = df.loc[i, 'Student']
23         ikke_bestatt_liste.append(navn)
24
25 # 3. Printer listen til slutt
26 print("Oversikt over karakterer:")
27 print(df.head())
28 print(f"\nStudenter som ikke bestod: {ikke_bestatt_liste}")

```